Частное профессиональное образовательное учреждение
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА»

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 4

По дисциплине: «Системное программирование»

Тема: «Копирование, перемещение и переименование»

Выполнил: студент
Группы: ИСиП-23-1
ФИО: Зыков Егор Тимурович

Проверил: преподаватель
Кузнецов А.Н.
Оценка: _____
Дата: «12» ноября 2026 г.

Сургут, 2026

1. Тема, цель и основные понятия

Тема: Копирование, перемещение и переименование.

Цель: Освоить безопасное копирование, переименование и перемещение собственных тестовых файлов, проверку исходного и целевого путей и подтверждение результата каждой операции.

Копирование создаёт дополнительный экземпляр данных и сохраняет источник. Переименование меняет имя пути в том же каталоге, а перемещение меняет расположение. copy2() также пытается сохранить доступные метаданные, но это не означает полного копирования всех особенностей файловой системы. Проверка назначения защищает учебные файлы от случайной перезаписи.

Средства реализации: Python 3 и стандартная библиотека pathlib, shutil.

2. Выполнение учебных заданий

2.1. Задание 1. Подготовить file_lab

Внутри Path.cwd() создаются incoming, backup и ready; в incoming программно записываются notes.txt и data.txt минимум с двумя строками. Исходная структура фиксируется до операций.

Фактический результат: Первый запуск создал file_lab/incoming/notes.txt и data.txt. В исходном выводе incoming содержит оба файла, backup и ready пусты; по коду каждый исходный файл получает две строки.

2.2. Задание 2. Скопировать notes.txt

shutil.copy2() копирует incoming/notes.txt в backup/notes_backup.txt. Проверка обоих путей после операции отличает копирование от перемещения.

Фактический результат: notes.txt скопирован в file_lab/backup/notes_backup.txt. Журнал первой операции показывает существование источника и назначения = True и «Проверка результата: True». Источник остался в incoming.

2.3. Задание 3. Переименовать data.txt

Path.rename() меняет incoming/data.txt на incoming/measurements.txt. Проверяются старый и новый пути: прежнее имя больше не должно ссылаться на файл.

Фактический результат: incoming/data.txt переименован в incoming/measurements.txt. Сразу после операции исходный путь существует = False, целевой = True, проверка результата = True.

2.4. Задание 4. Переместить measurements.txt

shutil.move() переносит incoming/measurements.txt в ready/measurements.txt. Контроль должен затрагивать оба каталога, а не только наличие целевого файла.

Фактический результат: measurements.txt перемещён из incoming в ready. После операции исходный путь отсутствует, ready/measurements.txt существует; проверка результата = True.

2.5. Задание 5. Добавить защитную проверку

Перед операцией проверяются exists() и is_file() источника и отсутствие целевого пути. При занятом целевом имени повторное копирование пропускается с сообщением. Результаты первого и повторного запусков сравниваются по журналам.

Фактический результат: При первом запуске копирование выполнено. Во втором получено сообщение «Целевой путь уже существует, операция пропущена» для notes_backup.txt. Исходники уже завершённой цепочки не воссозданы; переименование и перемещение пропущены из-за отсутствия исходных файлов. Это ожидаемые сообщения защиты, а не повторное выполнение операций.

2.6. Задание 6. Проверить file_lab

Через iterdir() выводится фактическое содержимое incoming, backup и ready. По нему определяется, какие файлы сохранились, какая копия создана и какой файл перемещён.

Фактический результат: После операций в file_lab/incoming остался notes.txt; в backup находится notes_backup.txt; в ready — measurements.txt. Второй запуск сохранил тот же состав.

3. Итоговая самостоятельная программа

Название: «Организатор тестовых файлов».

В workspace_ops созданы incoming, backup и ready, а также task.txt, config.txt и draft.txt с двумя строками каждый. По первому журналу task.txt скопирован в backup/task_backup.txt; config.txt переименован в settings.txt, после чего settings.txt перемещён в ready. Каждая операция дала проверку

True; побайтовое сравнение task.txt и копии дополнительно подтверждено текущими файлами. Итоговый incoming содержит draft.txt, memo_final.txt и task.txt, backup — task_backup.txt, ready — settings.txt. При повторном запуске целевая копия сохранена, отсутствующие исходники завершённых операций не создавались заново.

3.1. Полный исходный код main.py

```
"""Практическая №4. Организатор тестовых файлов. Зыков Е. Т., ИСиП-23-1."""

import argparse
from pathlib import Path
from shutil import copy2, move


def checkpoint(message, pause=False):
    print("\n=== " + message + " ===", flush=True)
    if pause:
        input("Enter — продолжить: ")


def create_source(path, text, later_paths=()):
    """Не воссоздаёт исходник, если цепочка уже дошла до следующего имени."""
    if path.exists() or path.is_symlink():
        print("Существующий путь сохранён:", path)
        return False
    for later in later_paths:
        if later.exists() or later.is_symlink():
            print("Исходник не воссоздан: следующий путь уже занят:", later)
            return False
    if not path.parent.is_dir():
        print("Родительский каталог отсутствует:", path.parent)
        return False
    try:
        with open(path, "x", encoding="utf-8", newline="\n") as file:
            file.write(text)
    except OSError as error:
        print("Ошибка создания:", error)
        return False
    print("Создан:", path)
    return True


def safe_operation(source, target, operation):
    if not source.exists() or not source.is_file() or source.is_symlink():
        print("Исходный файл не найден или не является обычным файлом:", source)
        return False
    if target.exists() or target.is_symlink():
        print("Целевой путь уже существует, операция пропущена:", target)
        return False
    if not target.parent.is_dir():
        print("Целевой каталог отсутствует:", target.parent)
        return False
    try:
        if operation == "copy":
            copy2(source, target)
        elif operation == "rename":
            source.rename(target)
        elif operation == "move":
            move(str(source), str(target))
        else:
            raise ValueError("Неизвестная операция: " + operation)
    except OSError as error:
        print("Операция не выполнена:", error)
        return False
    print(f"{operation}: {source} -> {target}")
```

```
    print("Исходный путь существует:", source.exists())
    print("Целевой путь существует:", target.exists())
    if operation == "copy":
        success = source.is_file() and target.is_file() and source.read_bytes() == target.read_bytes()
    else:
        success = not source.exists() and target.is_file()
    print("Проверка результата:", success)
    return success


def safe_copy(source, target):
    return safe_operation(source, target, "copy")


def safe_rename(source, target):
    return safe_operation(source, target, "rename")


def safe_move(source, target):
    return safe_operation(source, target, "move")


def show_folder(folder):
    print("\nСодержимое:", folder)
    if not folder.is_dir():
        print("Каталог не найден")
        return
    for item in sorted(folder.iterdir()):
        if item.is_file():
            print("[ФАЙЛ]", item.name)
        elif item.is_dir():
            print("[КАТАЛОГ]", item.name)


def prepare(base, source_name, copy_name, rename_name, final_name, extra=False):
    for name in ("incoming", "backup", "ready"):
        (base / name).mkdir(parents=True, exist_ok=True)
    incoming = base / "incoming"
    create_source(incoming / source_name, "Учебная задача Зыкова Егора.\nСтатус: подготовлена к копированию.\n",
                  (base / "backup" / copy_name,))
    create_source(incoming / rename_name, "Параметры учебной программы.\nРежим: безопасные операции.\n",
                  (incoming / final_name, base / "ready" / final_name))
    if extra:
        create_source(incoming / "draft.txt", "Черновик отчёта.\nПроверить результаты операций.\n")
        create_source(incoming / "memo.txt", "Памятка по работе.\nНе перезаписывать целевые файлы.\n",
                      (incoming / "memo_final.txt",))
    return incoming


def run_operations(base, source_name, copy_name, rename_name, final_name, pause=False, extra=False):
    incoming = prepare(base, source_name, copy_name, rename_name, final_name, extra)
    for name in ("incoming", "backup", "ready"):
        show_folder(base / name)
    checkpoint("Исходная структура: перед копированием", pause)
    safe_copy(incoming / source_name, base / "backup" / copy_name)
    checkpoint("После копирования: перед переименованием", pause)
    safe_rename(incoming / rename_name, incoming / final_name)
    checkpoint("После переименования: перед перемещением", pause)
    safe_move(incoming / final_name, base / "ready" / final_name)
    checkpoint("После перемещения", pause)
    if extra:
        safe_rename(incoming / "memo.txt", incoming / "memo_final.txt")
        checkpoint("Самостоятельное переименование memo завершено", pause)
    for name in ("incoming", "backup", "ready"):
        show_folder(base / name)
    checkpoint("Итоговая структура проверена", pause)
    return base


def main(pause=False):
```

```
    print("Рабочая папка:", Path.cwd())
    return run_operations(Path.cwd() / "workspace_ops", "task.txt", "task_backup.txt",
                          "config.txt", "settings.txt", pause=pause, extra=True)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Организатор тестовых файлов")
    parser.add_argument("--pause", action="store_true", help="ожидать Enter между операциями")
    main(pause=parser.parse_args().pause)
```

4. Самостоятельное изменение

В workspace_ops/incoming программно создан дополнительный memo.txt с двумя строками «Памятка по работе.» и «Не перезаписывать целевые файлы.». Выполнено безопасное переименование workspace_ops/incoming/memo.txt → workspace_ops/incoming/memo_final.txt. Первый запуск показал False для исходного пути, True для нового и проверку результата True; текущее состояние файлов совпадает с этим выводом. Во втором запуске memo.txt не воссоздан, так как memo_final.txt уже существует.

5. Подтверждение выполнения

Фактические результаты приведены в разделе выполнения заданий и таблице наблюдений. Источниками подтверждения служат реальные журналы запуска, сохранённые отдельно рядом с программами:

outputs\exercises_run1.txt

outputs\exercises_run2.txt

outputs\main_run1.txt

outputs\main_run2.txt

outputs\tests.txt

Скриншоты выполнения в этот отчёт не включены: по согласованному ограничению подтверждение представлено фактическими текстовыми результатами и отдельными журналами outputs. Это не является полным соответствием требованию исходного задания о скриншотах; изображения консоли и Проводника не подменялись искусственно созданными иллюстрациями.

Выдержка: outputs/main_run1.txt

```
Исходный путь существует: True
Целевой путь существует: True
Проверка результата: True
```

Выдержка: outputs/main_run1.txt

```
Исходный путь существует: False
```

Целевой путь существует: True
Проверка результата: True

Выдержка: outputs/main_run1.txt

Исходный путь существует: False
Целевой путь существует: True
Проверка результата: True

Выдержка: outputs/tests.txt

Ran 8 tests in 0.052s

OK

6. Таблица реальных наблюдений

Наблюдение	Фактическое значение	Пояснение
Исходный файл для копирования	workspace_ops/incoming/task.txt	После copy2() существует; исходные данные сохранены.
Путь копии	workspace_ops/backup/task_backup.txt	Существует; байты равны источнику.
Старое имя переименования	workspace_ops/incoming/config.txt	После первого rename() не существует.
Новое имя переименования	workspace_ops/incoming/settings.txt	Сразу после rename() существует; затем перенесено в ready.
Исходный путь перемещения	workspace_ops/incoming/settings.txt	После move() не существует.
Целевой путь перемещения	workspace_ops/ready/settings.txt	Существует после первого и повторного запусков.
Самостоятельная операция	incoming/memo.txt → incoming/memo_final.txt	Переименование внутри workspace_ops; старый путь False, новый True.

7. Контрольные вопросы и ответы

1. Чем копирование отличается от перемещения файла?

После копирования источник остаётся, а в назначении появляется копия. После успешного перемещения файл находится по новому пути, а старый путь освобождается.

2. Что происходит с исходным файлом после copy2()?

Исходный файл сохраняется на прежнем месте и с прежним содержимым; создаётся отдельная копия.

3. Что происходит с исходным путём после move()?

После успешного перемещения исходный путь больше не содержит перемещённый файл; проверка exists() для него возвращает False, если другой объект не создан заново.

4. Для чего используется Path.rename()?

Для переименования объекта. Если родительский каталог назначения тот же, меняется только имя; метод также способен менять путь в пределах поддерживаемой файловой системой операции.

5. Зачем перед операцией проверять destination.exists()?

Чтобы не перезаписать существующий объект и сообщить о занятом целевом имени. Это учебная защита, а не атомарная гарантия при конкурентном доступе.

6. Почему операции выполняются только внутри собственной рабочей папки?

Чтобы работать с специально созданными тестовыми данными, не затрагивая системные файлы, документы других пользователей и реальные рабочие материалы.

7. Как проверить, что переименование действительно выполнено?

Проверить отсутствие старого пути и наличие нового, затем сверить имя и при необходимости содержимое в каталоге.

8. Почему после копирования могут существовать два файла с одинаковым содержимым?

Копирование создаёт новый файл с теми же данными и не удаляет исходный. Одинаковое содержимое не делает два разных пути одним файлом.

8. Вывод

Выполнены копирование, переименование и перемещение тестовых файлов внутри рабочей папки. Копирование сохранило task.txt и создало равную ему копию, а переименование и перемещение освободили исходные пути. Проверки исходника и назначения предотвратили повторную перезапись при втором запуске. Самостоятельное переименование memo.txt в memo_final.txt подтверждено журналом и файловой структурой.